const qualifier

Type qualifiers

- const
- volatile

<typequalifier> Declaration ;

const int pi = 3.14;

volatile int x ;

```
main() {
    Const int var1 = 20;
    int var2;
    var2 = var1;
    Pf("%d ", var1 + var2); // 40
}
```

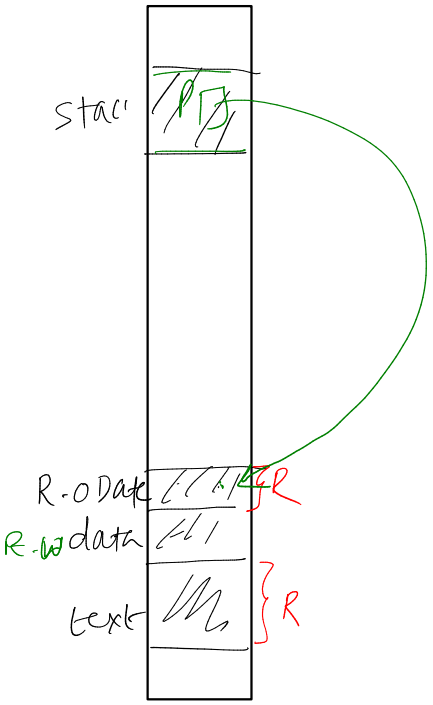
---

const float Pi = 3.14; // Read-only variable

```
main() {
    //const float Pi = 3.14;
    area = 3.14 * r * r;
    circumference = 2 * 3.14 * r;
}
```

Q. Is const-qualified variable (R.O. variable) modifiable?

A.
If we decided it to be constant, then why we are modifying?
if modification is the intention, then why qualify it to be const?



1.c

```
static void f1() {
    ≡
}

main() {
    f1();
    f2();
}
```

2.c

```
static void f1()
{
    ≡
}

f2() {
    ≡f1();
}
```

$ gcc 1.c 2.c ↵

→ Try the above one. use $nm a.out.

storage class in function

- extern (default)
- static

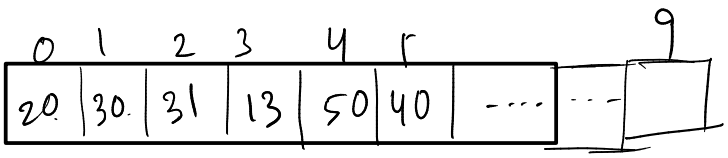
→ A static function has file scope

- ⇒ Restriction
- ⇒ overloading (multiple defn having same name)

W.A.P to re-order the Array of integers, so that

→ all prime numbers should be at the starting

I/P



O/P

31, 13, 20, 30, 50, 40 ---

```
int arr[10];
input(arr, 10); // random Ints between 2 - 40   (unique)
print(arr, 10);
reorder(arr, 10);
print(arr, 10);
```

```
void input(int *p, int n)
{
    int i;
    srand(getpid());
    for(i=0; i<n; i++)
    {
        p[i] = rand()%39+2      (0 ~ 38)
        if(checkRepeat(p, i)==1)
            i--;
    }
```